

noexit — 팀 공유 자료

TRUST404 트랙 1 제출 프로젝트 · 레포 github.com/ghwo336/T404 · 2026-09-19 · 이호재

한 줄 요약 — Solidity 소스(.sol)를 폴더째 넣으면, **컴파일러 · RPC · LLM · 네트워크 없이** 파일마다 **Benign / Malicious / Uncertain**을 판정하고 그 근거(파일·라인·코드 스니펫·설명)를 JSON과 CLI로 출력하는 **오프라인 정적 분석기**입니다. 토큰 컨트랙트에 숨어 있는 owner 권한(매도 차단, 블랙리스트, 무제한 수수료, 숨은 발행 등)을 찾는 데 특화돼 있습니다.

1. 대회 정보와 마감

항목	내용
대회	TRUST404 — Web3 보안 해커톤 (주최 ANAM145 · Blockchain Valley)
트랙	기술 트랙 1: 스마트 컨트랙트 악성 판별
온라인 제출 마감	9월 20일 (일) 23:59 — 제출물: 피치덱 · 시연 영상 · GitHub
오프라인 데모데이	9월 29일 (화) 17:30-22:00, 드림플러스 강남 (서초구 강남대로 311)
시상	Grand 200만 · Technical Excellence 100만 · Ecosystem Innovation 50만 · Security Impact 50만, 수상팀 전원 후속 협업 논의
연락	Discord (공개 샘플 배포), dev@anam145.com, trust404.co.kr

트랙 1이 실제로 요구하는 것 (trust404.co.kr/tracks)

- 입력: .sol 파일. 디렉토리 단위 배치 처리 필수
- 출력: Benign / Malicious / Uncertain 판정 + 코드 위치 + 근거. JSON 등 기계 판독 가능 포맷
- **채점은 네트워크가 차단된 환경** → 외부 API · LLM · RPC · 시뮬레이션(Tenderly, eth_call 등) 전부 사용 불가
- 비대화형 CLI 진입점 필수 (웹 UI는 부가)
- 심사 기준: ① 공개 샘플 + 비공개 테스트셋 분류 정확도 ② 키워드 매칭이 아닌 **로직 흐름·권한 구조** 기반 분석 ③ 근거의 타당성

방향이 바뀐 이유 — 처음 아이디어는 "트랜잭션 보내기 전에 위험한 컨트랙트인지 감지하는 SDK"였습니다. 그런데 트랙 1은 온체인 시뮬레이션이 아니라 **오프라인 소스 분석기**를 채점합니다. 그래서 코어를 정적 분석 CLI로 잡고, 원래의 SDK/지갑 연동은 "같은 엔진 위에 얹을 확장"으로 포지셔닝했습니다. **현재 엔진 코드에는 네트워크 호출이 하나도 없습니다** (벤치마크 스크립트만 소스 다운로드에 Etherscan을 쓰고, 채점 대상인 scan 명령과는 분리돼 있음).

2. 지금까지 만든 것

구성요소	내용	위치
분석 엔진	TypeScript. @solidity-parser/parser(순수 JS, solc 불필요, pragma 0.4~0.8 파싱)로 AST를 만들고 룰 17종 실행. import 해석(상	src/

	대경로 · node_modules · lib · remappings.txt). Node와 브라우저 양쪽에서 동작	
CLI	scan 명령. pretty / json / sarif / summary 출력, --out, --fail-on	src/cli.ts → dist/cli.js
샘플 + 회귀 테스트	악성 17 · 정상 8 · 불확실 1 (멀티파일, 식별자 난독화 케이스 포함). npm test 26/26	samples/, test/
실제 벤치마크	학술 논문(Pied-Piper, ACM TOSEM 2022)이 라벨링한 이더리움 백도어 컨트랙트 188개 + 블루칩 토큰 29개. 결과표 bench/RESULTS.md	bench/
웹 UI	단일 HTML, 더블클릭으로 열림. .sol 파일/폴더 드롭 → 판정 · 근거 · 코드 하이라이트. 네트워크 없음. 시연 영상용	web/noexit.html
피치덱	12장 (문제 → 접근 → 룰 → 출력 예시 → 결과 → 한계 → 로드맵). PDF/PPTX 다운로드 가능	Claude 아티팩트 (호재 공유)
README	설치 · 실행 · 출력 스키마 · 룰 표 · 벤치마크 · 한계	README.md

3. 동작 원리 (팀원용 요약)

핵심 질문은 하나입니다 — "누가(어떤 권한으로) 무엇을 쓸 수 있고, 토큰 전송 경로가 그 값을 어떻게 사용하는가?" 키워드나 함수 이름을 찾는 게 아니라 이 흐름을 추적합니다.

1. **컨트랙트 모델** — 상속을 평탄화하고, 상태변수의 **역할**을 이름이 아니라 쓰임새로 정합니다. 예: `balanceOf()`가 리턴하는 매핑 = 잔고, `createPair()` 결과나 전송 경로에서 `to/from`과 비교되는 주소 = DEX pair, 생성자에서 `msg.sender`를 받거나 `modifier`에서 `msg.sender`와 비교되는 주소 = owner. `return msg.sender`만 하는 함수는 이름이 뭐든 `_msgSender` 별칭으로 인식. 그래서 변수명을 전부 `_q1`, `_q2`로 바꿔도 결과가 같습니다.
2. **전송 경로** — `transfer/_transfer/_update...` 계열 + 잔고 매핑을 쓰는 함수 + 그들이 호출하는 내부 함수의 폐포. 토큰의 위험 로직은 결국 여기 있어야 합니다.
3. **룰 17종** — "권한 함수의 쓰기"가 "전송 경로의 require / if-revert / 수수료 산술 / 잔고 쓰기"로 이어지는지 확인합니다. 수수료는 로컬 변수, 상태→상태 대입, 내부 함수 인자→파라미터 경유까지 데이터플로우로 따라갑니다.
4. **판정** — 심각도 × 신뢰도. critical 1개 또는 high 2개 → Malicious, high 1개 또는 medium 2개 → Uncertain, 나머지 Benign.

룰 목록 (무엇을 확인하는지)

룰 ID	확인하는 것
SELL_RESTRICTION · SELL_FEE_UNCAPPED	전송 경로에서 pair 방향(매도)만 다르게 취급되는 분기가 있는지, 매도 분기의 수수료 변수에 owner 세터 상한이 없는지
BLACKLIST_GATE · TRADING_GATE	권한 함수가 쓰는 주소 목록/플래그가 전송의 차단 조건으로 쓰이는지. 표준 OZ Pausable은 medium, 커스텀 플래그는 high
UNCAPPED_FEE · OWNER_LIMIT_TO_ZERO	전송 산술에 닿는 uint의 세터에 상한이 없는지, 전송량 제한 변수의 세터에 하한이 없는지
HIDDEN_MINT · OPEN_MINT · OWNER_MINT	생성자 밖에서 잔고+공급을 늘리는 함수 (이름 위장 / 무권한 / 캡 유무로 등급 구분)
BALANCE_MANIPULATION	전송 경로 밖의 권한 함수가 잔고 매핑을 직접 쓰는지 (같은 슬롯 add/sub 갱신과 덮어쓰기 구분)
APPROVAL_BYPASS	타인의 allowance를 쓰는 함수, allowance를 참조하지 않는 <code>transferFrom</code> , 특정 호출자에게만 건너뛰는 분기
FAKE_RENOUNCE · HIDDEN_OWNER_TRANSFER · OPEN_OWNER_TAKEOVER	owner 변수가 표준 함수 밖에서 바뀌거나, <code>renounce</code> 가 실제로 owner를 비우지 않는지
EXTERNAL_TRANSFER_HOOK	전송 경로가 owner가 교체할 수 있는 외부 컨트랙트에 판단을 위임하는지
SELFDESTRUCT · DELEGATECALL · OPEN_DRAIN · PRIVILEGED_WITHDRAW · TX_ORIGIN_AUTH · UNRESOLVED_BASE	위험 연산과 자금 출구, 분석 못 한 상속 부모에 대한 안내

Malicious finding에는 attackPath(어떤 권한 함수가 방아쇠이고 어느 줄에서 막히는지 순서), Benign 판정에는 checks[] (12개 검사 항목의 pass/fail과 근거)가 함께 나옵니다. 심사 기준 ③ 대응입니다.

4. 실행 방법

```
git clone https://github.com/ghwo336/T404.git && cd T404
npm install && npm run build

node dist/cli.js scan ./samples --out report.json      # 폴더 재귀 스캔 + JSON 리포트
node dist/cli.js scan ./samples -f summary            # 채점용 한 줄 요약 (verdict / score / file)
node dist/cli.js scan some.sol -v                    # 근거·설명 전부 출력
npm test                                               # 샘플 26개 회귀 테스트
npm run bench -- --offline                            # 캐시된 실제 컨트랙트 벤치마크 (Etherscan 키 불필요)
```

웹 UI: web/noexit.html을 브라우저로 열고 "Load bundled samples" 또는 .sol 폴더 드롭. 네트워크 없이 동작합니다.

5. 결과

26/26 자체 샘플 회귀 테스트	98.7% precision (실제 217개)	80.3% recall · Uncertain 포함 91.5%	0.886 F1
------------------------------	-------------------------------------	--	--------------------

라벨 \ 판정	Malicious	Uncertain	Benign
실제 백도어 188개 (Pied-Piper, ACM TOSEM 2022)	151	21	16
블루칩 토큰 29개 (WETH · UNI · LINK · DAI · AAVE · COMP · SHIB · PEPE ...)	2	4	23

솔직한 해석 (발표 때 그대로 쓰면 됨)

- 블루칩 "오타" 2개(PEPE, LDO)는 코드 기준으로는 정탐입니다 — 실제로 owner/controller 권한이 소스에 있습니다. "우리는 코드를 본다"는 강점으로 설명.
- OpenZeppelin Pausable과 캡 없는 owner mint는 **의도적으로 medium** (1INCH, SUSHI, YFI, ENS 같은 정상 토큰도 가지고 있음). 커스텀 플래그는 high.
- 미탐 16개는 상당수가 데이터셋 노이즈 (전송 경로가 읽지 않는 죽은 freeze 함수, 잘못 라벨된 체인링크 Oracle.sol, 양쪽 리스트에 다 있는 MKR).
- 한계: 인라인 어셈블리(store) 트릭과 Vyper는 범위 밖. 입력에 없는 import(@openzeppelin 등)는 표준 이름으로 가정하고 UNRESOLVED_BASE로 표시.

6. 남은 일과 역할 분담 (제안)

#	할 일	메모
1	Luma 등록 + Discord 입장 → 공개 샘플 수령	비공개 테스트셋 스타일을 알아야 정확도가 결정됨. 받는 즉시 scan 돌려서 결과 공유
2	GitHub push 확인 (T404 레포)	맥 터미널에서 git push -u origin master. README/텍의 레포 URL은 이미 T404로 맞춰둠
3	시연 영상 1~2분	① 웹 UI에서 샘플 로드 → Malicious 하나 클릭 → 코드 하이라이트·체크리스트 ② 터미널에서 scan samples --out report.json ③ 와이파이 끈 채로 돌아가는 장면 한 컷
4	피치덱 최종 검토	12장. 스피커 노트에 장별 멘트 있음. 팀명/팀원 이름 표지에 추가 필요
5	데모데이 발표자 · Q&A 담당 정하기	예상 질문: "키워드 매칭 아닌가?" → 난독화 샘플 · 데이터 플로우, "오프라인 맞나?" → 의존성 2개 · 네트워크 호출 없음, "정확도?" → 벤치마크 표
6	(선택) Dockerfile	채점 환경에 node가 없을 리스크 제거. 30분 작업

7. 자주 나올 질문

Q. Tenderly / eth_call 같은 건 안 쓰나요? — 안 씁니다. 엔진은 소스 텍스트 → AST → 룰 뿐이고, 의존성은 파서와 CLI 옵션 파서 둘입니다. `grep -rn "fetch\|http\|rpc" src/` 하면 아무것도 안 나옵니다.

Q. 여러 파일로 나뉜 프로젝트도 되나요? — 됩니다. import를 입력 폴더 안에서 따라가서 하나의 모델로 합칩니다. 토큰 파일은 깨끗하고 import된 라이브러리 파일에 문제가 있는 샘플도 그 파일의 줄 번호를 찍어 잡습니다.

Q. 왜 Uncertain이 있나요? — "owner가 전송을 멈출 수 있음"처럼 정상 프로젝트에도 흔한 중앙화 요소는 확신을 갖고 Malicious라고 하지 않습니다. 근거는 똑같이 출력되므로 심사 쪽에서 판단할 수 있습니다.

Q. 어디까지가 우리 코드인가요? — 파서 라이브러리를 제외한 `src/`, `web/`, `bench/`, `samples/` 전부 이번 해커톤에서 작성했습니다.